Introduction to Asynchronous JavaScript

Presented by:

Cristian Camilo Cortes Ortiz

Codecamp academy

Table of contents

Audience 2
Purpose 2
Introduction 2
What is Asynchronism? 3
Main Concepts 4
- Single-Threaded 4
- Blocking 4
- Non-Blocking 5
- Synchronous 5
- Asynchronous 5
- Event Loop 5
- Callbacks 6
- Promises 6
- Async / Await 6

Evolution of JavaScript 6
Event Loop 7
- Call Stack 7
- Memory Heap 7
- Task Queue 8
- Microtask Queue 8
- Web APIs and Node.js APIs 8

Callbacks 10
XMLHttpRequest 12
Promises 13
- Creating a Promise 13
- Consuming a Promise 14
- Combining Promises 14

Async / Await 15
Best Practices 16
Conclusion 16

Audience

This manual contains technical terminology and is intended for software development students, instructors, and developers who want to learn the fundamentals of asynchronous programming in JavaScript. It covers the basic concepts needed to handle asynchronous tasks, from the event loop to callbacks, promises, and async/await. Readers should be familiar with basic JavaScript syntax, functions, and objects.

Purpose

The purpose of this manual is to give readers a clear understanding of the basic concepts for handling concurrency and asynchronism in the execution of tasks by the event loop of the JavaScript programming language. By the end of this document, the reader will be able to:

- Explain the difference between synchronous and asynchronous execution.
- Describe how the event loop, the call stack, and the queues work together.
- Use callbacks and XMLHttpRequest to perform asynchronous requests.
- Create and consume promises.
- Write asynchronous code with async/await.
- Recognize common problems such as callback hell and apply good practices.

Introduction

JavaScript handles concurrency and asynchronism through an event loop, which the language engine implements by means of stacks and queues of tasks that order the events executed by the running process. Understanding this model is essential to write code that does not freeze the user interface or the server while it waits for slow operations such as network requests, timers, or file access.

This document starts with the main concepts, explains how the event loop works, and then presents the three main tools for handling asynchronous tasks in JavaScript: callbacks, promises, and async/await.

What is Asynchronism?

By default, programming languages are synchronous, which means that tasks must be executed one by one. As technologies evolved, concepts, libraries, and resources were introduced that give programming languages the ability to work asynchronously.

JavaScript is a synchronous programming language by default, and it has only one thread, that is, a single thread of execution. Different documentation describes it in another way: JavaScript is a synchronous, non-blocking language with an event loop that handles concurrency using a single thread for its input and output interfaces. This definition helps to understand how JavaScript works and which challenges it faces. Figure 1 compares both execution models.

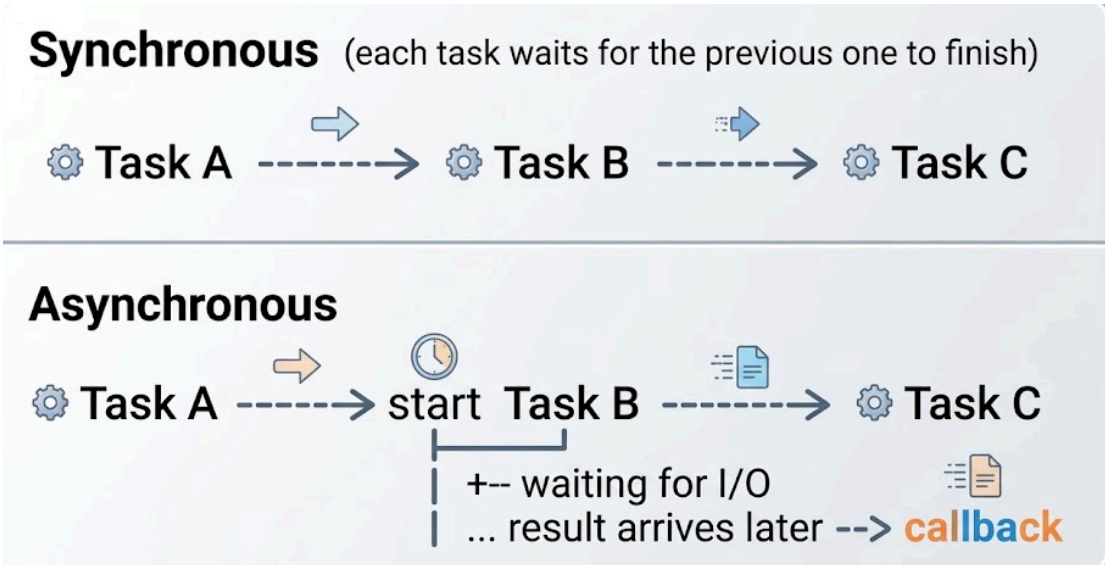


Figure 1. Synchronous and asynchronous execution of three tasks

Table 1 summarizes the differences between both models.

Aspect	Synchronous	Asynchronous
Execution	Task by task, in order	Tasks can be resolved later
Waiting	Each task waits for the previous one	The application keeps running
Response	Immediate	Deferred (callback or promise)
Typical use	Simple calculations	Network requests, timers, files

Table 1. Synchronous versus asynchronous execution

Main Concepts

The following concepts are the foundation for understanding asynchronous programming in JavaScript.

Single-Threaded

Even with multiple processors in the CPU, JavaScript runs code in a single thread, called the main thread. Only one instruction is executed at a time.

Blocking

Blocking means that the execution of a task does not return control until the task is completed. Therefore, a task can block the rest of the application, for example, when the alert function is used.

Non-Blocking

Non-blocking means that the execution of a task immediately returns control, so the application can continue running without having to stop. The result is tracked afterwards, returning the data or handling an error as the case may be. Figure 2 contrasts both behaviors.

```
// Blocking: the thread stops until the user closes the dialog
alert('Hello');
console.log('This line waits for the alert');

// Non-blocking: control returns immediately
setTimeout(() => console.log('Runs later'), 1000);
console.log('Runs first');

```

Figure 2. Blocking call versus non-blocking call

Synchronous

Synchronous means that tasks executed sequentially must wait for the previous task to complete before the next one can continue.

Asynchronous

Asynchronous means that the tasks being executed can be resolved later, which makes it possible to defer the response and continue running the application.

Event Loop

To handle concurrency, JavaScript uses an event-based model. The event loop is a design pattern that waits for and dispatches events or messages in a program.

Callbacks

A callback is a function that is passed as an argument to another function and is invoked when needed.

Promises

Promises are non-blocking objects that work asynchronously and will fulfill a 'promise'. That promise may be resolved now, in the future, or never.

Async / Await

Async/await allows an asynchronous, non-blocking function to be structured in a way very similar to a traditional function.

Evolution of JavaScript

As JavaScript acquired new features and functionalities over the years, it became capable of handling many tasks at the same time without blocking the program. It is important to clarify that the language was conceived as synchronous and still executes code in a single thread. What changed is that the runtime environments, such as browsers and Node.js, provide tools designed to work with asynchrony inside the project.

Based on the above, the definition of the JavaScript programming language changes slightly and can be stated as follows: JavaScript is an asynchronous, non-blocking programming language, with an event loop to handle concurrency, implemented with a single thread for its input and output interfaces.

Event Loop

The event loop is the mechanism that allows JavaScript to perform non-blocking operations despite having a single thread. It is made up of several elements that work together, as shown in Figure 3.

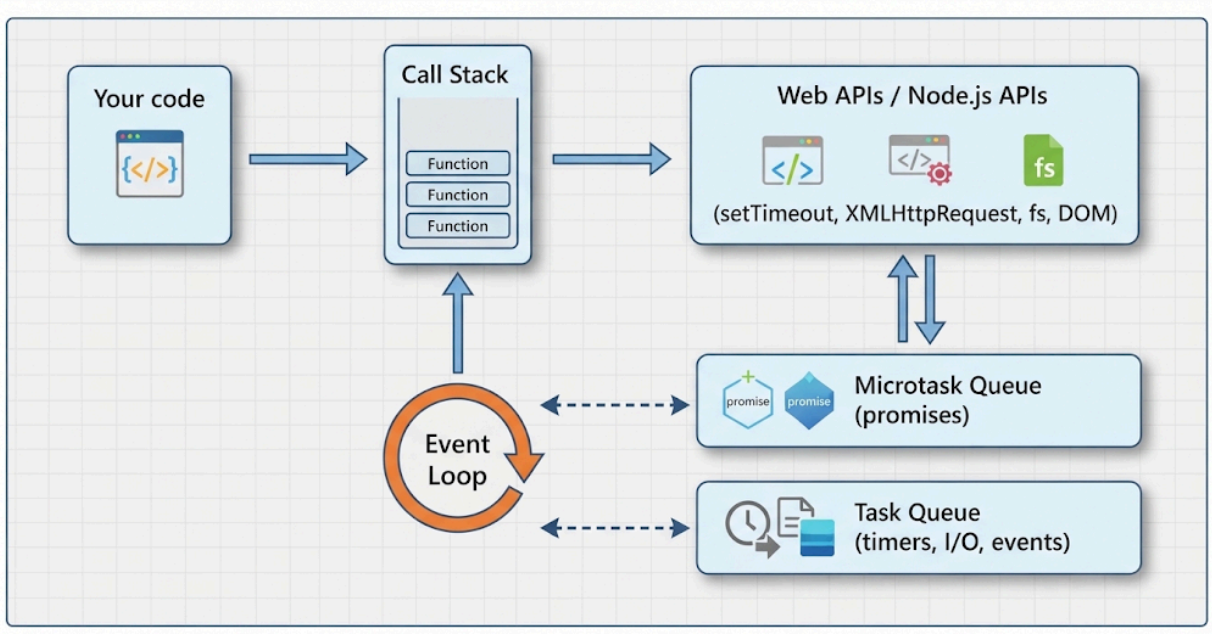


Figure 3. Elements of the JavaScript event loop

Call Stack

The call stack organizes the instructions of the program in order, which makes it possible to know how they will be executed. It works as a stack: the last function that enters is the first one to leave. The stack must never be blocked, and it must be empty for the program to continue with the logic of the queues.

Memory Heap

Objects are allocated in a heap, an unstructured region of memory, while they wait to be used.

Task Queue

The task queue handles concurrency directly. Tasks that are ready to be executed, such as the callbacks of timers or events, are added here and wait to move to the stack. The stack must be empty for the next task to enter.

Microtask Queue

Promises are executed in a different way and with a higher priority. Their callbacks are placed in the microtask queue, which is emptied before the next task of the task queue is processed.

Web APIs and Node.js APIs

JavaScript was created inside the browser to handle interactions with the user, which involve elements such as:

- setTimeout.
- XMLHttpRequest.
- FileReader.
- DOM.

After the creation of Node.js, JavaScript also interacts with server-side elements such as:

- File system (fs).
- HTTP and HTTPS.

With all the previous elements, the event loop can be defined as a particular task within the whole logic. It is in charge of moving tasks from the queues to the stack, only if the stack is empty. Figure 4 shows the resulting execution order.

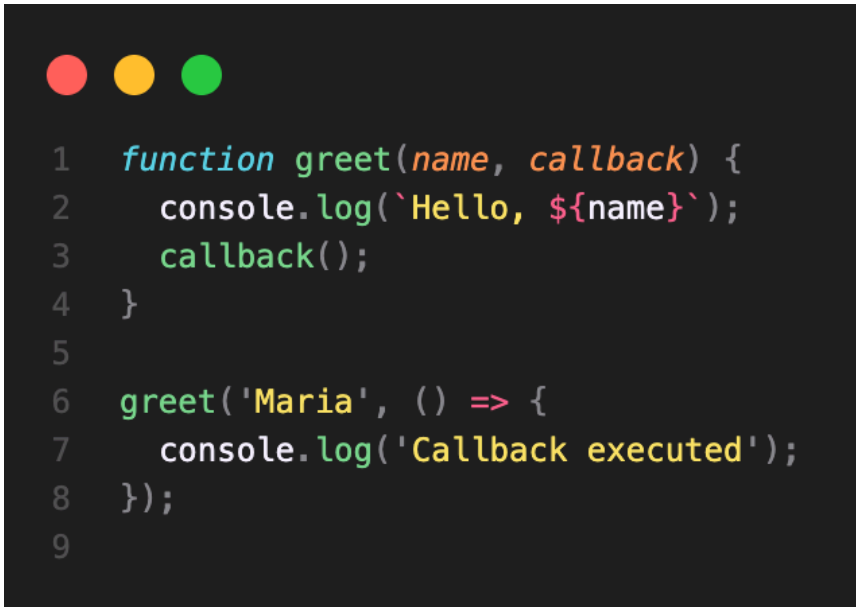
```
1  console.log('Start');
2  setTimeout(() => console.log('Timeout'), 0);
3  Promise.resolve().then(() => console.log('Promise'));
4  console.log('End');
5
6  // Output:
7  // Start
8  // End
9  // Promise
10 // Timeout
11
```

Figure 4. Execution order of synchronous code, promises, and timers

Notice that the promise callback runs before the timer, even though the timer was set to zero milliseconds: microtasks have priority over the task queue.

Callbacks

A callback is a function passed as an argument to another function, which invokes it when needed, for example, when an operation finishes. Callbacks were the first mechanism JavaScript used to handle asynchronous tasks. Figure 5 shows a simple example.



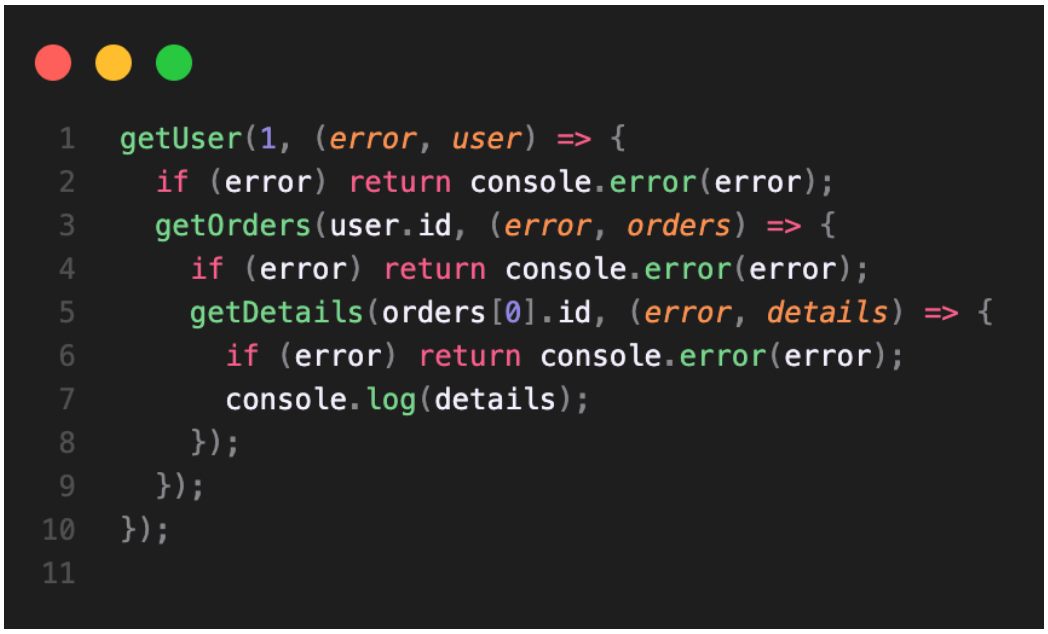
```
function greet(name, callback) {
  console.log(`Hello, ${name}`);
  callback();
}

greet('Maria', () => {
  console.log('Callback executed');
});

```

Figure 5. Function that receives and invokes a callback

In Node.js, callbacks follow the error-first convention: the first parameter is reserved for an error and the second one for the result. The main drawback appears when several asynchronous operations depend on each other, because the callbacks are nested one inside another and the code grows horizontally and becomes hard to read and maintain. This problem is known as callback hell, shown in Figure 6.



```
getUser(1, (error, user) => {
  if (error) return console.error(error);
  getOrders(user.id, (error, orders) => {
    if (error) return console.error(error);
    getDetails(orders[0].id, (error, details) => {
      if (error) return console.error(error);
      console.log(details);
    });
  });
});

```

Figure 6. Nested callbacks known as callback hell

XMLHttpRequest

XMLHttpRequest (XHR) is a browser API used to make HTTP requests asynchronously, without reloading the page. It reports the progress of the request through the readyState property, whose values go from 0 (request not initialized) to 4 (operation completed). When the state is 4 and the HTTP status is 200, the response is available in the responseText property. Figure 7 wraps a request in a function that uses an error-first callback.

```
function fetchData(url, callback) {
  const xhr = new XMLHttpRequest();
  xhr.open('GET', url, true);
  xhr.onreadystatechange = () => {
    if (xhr.readyState !== 4) return;
    if (xhr.status === 200) {
      callback(null, JSON.parse(xhr.responseText));
    } else {
      callback(new Error(`Request failed: ${xhr.status}`));
    }
  };
  xhr.send();
}

fetchData('https://jsonplaceholder.typicode.com/todos/1', (error, data) => {
  if (error) return console.error(error);
  console.log(data.title);
});

```

Figure 7. HTTP request with XMLHttpRequest and a callback

Although XMLHttpRequest is still supported, it has largely been replaced by the Fetch API, which is based on promises.

Promises

A Promise in JavaScript is an object that represents a value that may be available now, in the future, or never. It allows handlers to be associated with the eventual success or failure of an asynchronous operation. It is similar to synchronous methods that return values immediately, but instead of returning the final value, an asynchronous function returns a promise that provides the value later.

A Promise can be in one of three states, summarized in Table 2.

State	Meaning
pending	Initial state, neither fulfilled nor rejected
fulfilled	The operation completed successfully
rejected	The operation failed

Table 2. States of a Promise

Creating a Promise

A promise is created with the Promise constructor, which takes an executor function with two parameters: resolve (to fulfill the promise) and reject (to reject it). Figure 8 defines a function that returns a promise resolved after a delay.

```
const wait = (ms) =>
  new Promise((resolve, reject) => {
    if (ms < 0) return reject(new Error('ms must be positive'));
    setTimeout(() => resolve(`Waited ${ms} ms`), ms);
  });

```

Figure 8. Creating a promise with the Promise constructor

Consuming a Promise

A promise is consumed with the then, catch, and finally methods. The then method receives the result when the promise is fulfilled, catch handles the error when it is rejected, and finally runs in both cases, as shown in Figure 9. Each method returns a new promise, which makes it possible to chain operations in sequence and avoid the nesting of callbacks.

```
wait(1000)
  .then((message) => console.log(message))
  .catch((error) => console.error(error.message))
  .finally(() => console.log('Done'));

```

Figure 9. Consuming a promise with then, catch, and finally

Combining Promises

Promise.all receives a list of promises and returns a new promise that is fulfilled with the list of results when all of them are fulfilled, or rejected as soon as one of them is rejected. Promise.allSettled always waits for every promise and reports the outcome of each one, as shown in Figure 10.

```
Promise.all([wait(500), wait(1000)])
  .then((results) => console.log(results))
  .catch((error) => console.error(error));

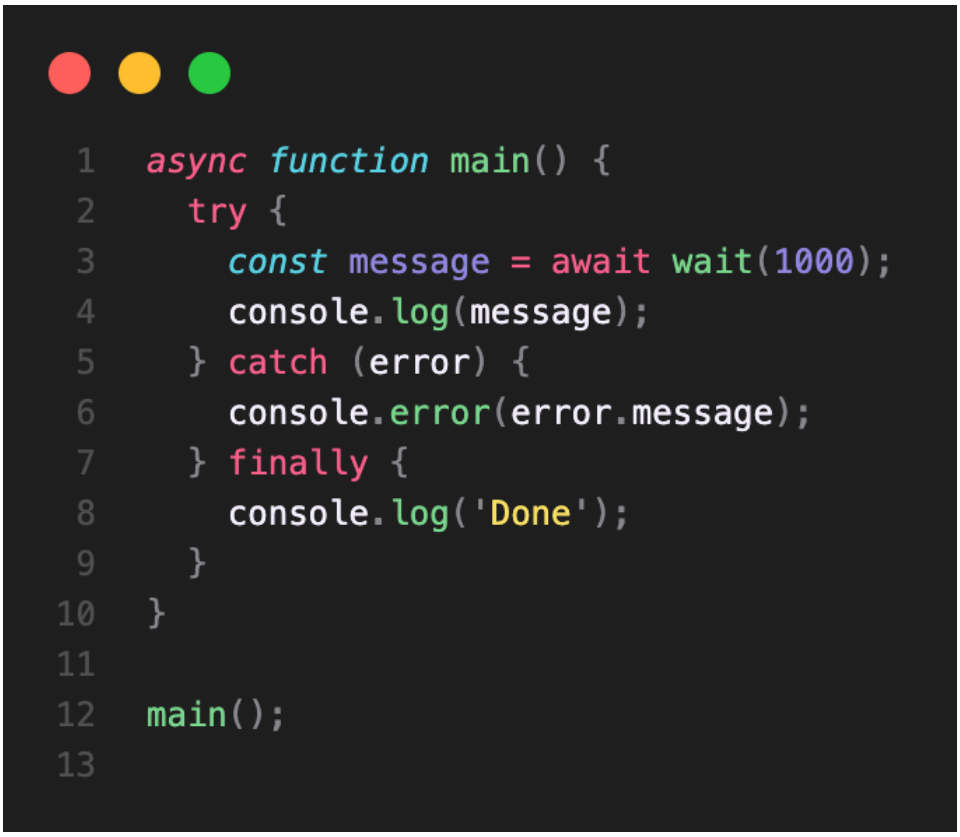
// Promise.all rejects as soon as one promise rejects
// Promise.allSettled always waits for every promise

```

Figure 10. Running several promises in parallel with Promise.all

Async / Await

Async/await is syntax built on top of promises that allows asynchronous code to be written in a way very similar to traditional synchronous code. A function declared with the async keyword always returns a promise, and the await keyword pauses the execution of that function, not of the whole program, until the promise is settled. Errors are handled with the usual try/catch/finally structure, as shown in Figure 11.



```
async function main() {
  try {
    const message = await wait(1000);
    console.log(message);
  } catch (error) {
    console.error(error.message);
  } finally {
    console.log('Done');
  }
}

main();
```

Figure 11. Asynchronous function written with async/await

Best Practices

The following recommendations help keep asynchronous code reliable and easy to maintain:

- Prefer promises and async/await over deeply nested callbacks.
- Always handle errors with catch or try/catch to avoid unhandled rejections.
- Never block the main thread with long synchronous operations.
- Run independent operations in parallel with Promise.all instead of awaiting them one by one.
- Keep functions small and return promises so they can be composed.
- Remember that microtasks run before timers when reasoning about the execution order.

Conclusion

Asynchronism allows JavaScript to remain responsive while it waits for slow operations, even though it runs on a single thread. The event loop, together with the call stack, the task queue, and the microtask queue, coordinates when each piece of code is executed. Callbacks were the first solution, promises improved readability and error handling, and async/await offers a syntax close to synchronous code. Mastering these tools is essential to build modern web applications and Node.js services.